Accessibility and visual testing: interview questions, answered with this project

Questions about accessibility testing (WCAG, axe-core, keyboard, screen readers) and visual regression testing that come up in SDET, QA lead and front-end quality interviews. Each answer ends with an **In the project** pointer. Links go to github.com/niranjankagri/accessibility-visual-testing.

Versions: Playwright 1.64, @axe-core/playwright 4.13 (axe-core 4.13), TypeScript 6.0, Node 24.

Contents

1. Accessibility basics (Q1-Q12)
2. Automated scanning with axe-core (Q13-Q26)
3. Keyboard and focus (Q27-Q37)
4. Forms, errors and dynamic content (Q38-Q44)
5. Contrast, zoom and motion (Q45-Q50)
6. Visual regression testing (Q51-Q66)
7. Testing the tests, process and scenarios (Q67-Q78)

1. Accessibility basics

Q1. What is WCAG?

The Web Content Accessibility Guidelines from the W3C: testable success criteria grouped under four principles (perceivable, operable, understandable, robust). Each criterion has a level: A (minimum), AA (the usual legal and contractual target) or AAA (enhanced, rarely required for a whole site). Version 2.2 is the current one.

In the project: the axe scans use the WCAG 2.0, 2.1 and 2.2 A and AA tags (axe.ts).

Q2. What did WCAG 2.2 add?

Nine success criteria, among them Focus Not Obscured (2.4.11, AA), Dragging Movements (2.5.7, AA), Target Size Minimum of 24 × 24 CSS pixels (2.5.8, AA), Consistent Help (3.2.6, A), Redundant Entry (3.3.7, A) and Accessible Authentication (3.3.8, AA). It also removed 4.1.1 Parsing, which modern browsers made obsolete.

In the project: the `wcag22aa` tag adds the axe rules for the new criteria (for example target size).

Q3. Why does accessibility matter beyond "it is the right thing"?

Many people have a permanent, temporary or situational disability; accessible products reach more customers. In the EU the European Accessibility Act applies to many consumer products and services since June 2025, and public sector sites have had obligations longer (EN 301 549 points to WCAG). Contracts often require WCAG 2.1 or 2.2 AA. And accessible markup (labels, roles, names) also makes UI tests more robust.

In the project: the app has an accessibility statement, as such laws require.

Q4. Who benefits from which fixes?

Screen reader users need names, roles, structure and announcements. Keyboard users need focus order, visible focus and no traps. Low-vision users need contrast, zoom and reflow. Users with vestibular disorders need reduced motion. Users with cognitive disabilities need clear errors and consistent layouts. The "who it hurts" column in the README's defect table makes this concrete.

In the project: each planted defect has a story naming who it hurts (defects.ts).

Q5. What is the "first rule of ARIA"?

Do not use ARIA if a native HTML element already has the semantics and behaviour you need. A `<button>` is focusable, has the button role and reacts to Enter and Space; a `<div role="button">` needs all of that added by hand. ARIA changes what assistive technology is told, not how the element behaves.

In the project: native buttons, labels, `fieldset` / `legend`, `<table>` with `<caption>` and `<dialog>`; ARIA only where HTML has no equivalent (`aria-describedby`, `aria-invalid`, `aria-current`).

Q6. What is an accessible name and where does it come from?

What a screen reader announces for an element: from `aria-labelledby`, `aria-label`, a `<label>`, the element's text content or `alt`, in that order of precedence. A button with only an icon and no name is announced as "button", which tells the user nothing.

In the project: visually hidden text gives each "Cancel" button a unique name ("Cancel appointment on 12 November"); the tests find buttons by that name.

Q7. Why do role-based locators help both UI and accessibility testing?

`getByRole('button', { name: 'Book visit' })` only finds the element if it has the right role and accessible name. A test written that way fails when the name disappears, so every UI test becomes a small accessibility check, and it does not break when a CSS class changes.

In the project: the tests use `getByRole` and `getByLabel` throughout (keyboard.spec.ts).

Q8. What is a landmark, and why do landmarks matter?

Regions such as `header` (banner), `nav`, `main`, `aside` and `footer` (contentinfo). Screen reader users jump between them like chapters. One `main` per page and content inside landmarks are axe best-practice rules.

In the project: the layout has banner, navigation, main, complementary (chat) and contentinfo.

Q9. How do headings help?

Screen reader users often navigate by headings, and many list them to get an overview. One `h1` naming the page and levels without gaps give a usable outline.

In the project: every page has one `h1`; axe's `heading-order` and `page-has-heading-one` rules run.

Q10. What is the difference between conformance and "no axe findings"?

Conformance means every applicable success criterion is met, which needs human judgement (is the alt text meaningful? does the focus order make sense?). A clean axe run means the automatable rules found nothing on the pages and states scanned.

In the project: the README says a clean run is evidence, not a conformance claim.

Q11. How do you test with a screen reader?

NVDA with Firefox or Chrome on Windows, VoiceOver with Safari on macOS and iOS, TalkBack on Android. Navigate by headings, landmarks and form fields; complete the main tasks; listen to what is announced on errors and confirmations. It is slow, so it is done for the main journeys before releases, not on every commit.

In the project: manual, by design; the accessibility statement says the team tests with a screen reader before each release.

Q12. Where does accessibility testing fit in a team's process?

Acceptance criteria in stories, design review of contrast and focus states, linting and component scans while building, page scans and keyboard tests in CI, manual screen reader checks before releases, and user testing with people with disabilities. Fixing it in design is cheapest.

In the project: the CI part (ci.yml).

2. Automated scanning with axe-core

Q13. What is axe-core?

Deque's open-source rules engine. It runs inside the page, checks the DOM and computed styles against its rules and returns violations, passes, incomplete results and inapplicable rules. Playwright, Cypress, Selenium, Jest and browser extensions all use it.

In the project: `@axe-core/playwright` with `AxeBuilder` (axe.ts).

Q14. How much does automated scanning find?

A part of the problems. Deque has published a figure of about 57% of issues by volume in their audits, but many success criteria cannot be decided by a machine at all (meaningful alt text, logical reading order, useful error text). Treat a scan as a fast safety net, not as the test.

In the project: that is why there are keyboard, focus and layout tests next to the scans.

Q15. Why does axe report "incomplete" results?

When it cannot decide, for example the contrast of text over an image, a gradient or a dialog backdrop. axe aims for no false positives, so uncertain cases are not violations. They need a person.

In the project: incomplete rules do not fail the test; the summary lists them ("rules axe could not decide": the open dialog's backdrop).

Q16. Which rule tags do you run, and why best practices too?

`wcag2a`, `wcag2aa`, `wcag21a`, `wcag21aa`, `wcag22aa` for the legal target, plus `best-practice` (headings, landmarks, positive `tabindex`). Best practices are not WCAG failures, but in a new app they are cheap to follow and catch real problems early. In a legacy app I would start with WCAG tags only.

In the project: `AXE_TAGS`; the positive `tabindex` defect is also caught by the best-practice `tabindex` rule.

Q17. Scan the page once, or more often?

In every state a user can reach: after opening a dialog, after a failed submit, after a success message, in dark mode. A page load shows only the first state; many bugs live in the others.

In the project: scans of the form with errors, the confirmation, the open dialog, and every page in dark mode (axe.spec.ts).

Q18. How do you scan only part of a page?

`AxeBuilder.include(selector)` and `exclude(selector)`. Useful for a component, a dialog, or excluding a third-party frame you test elsewhere. Excluding to make a test pass is the wrong use.

In the project: the dialog scan uses `include: '#cancel-dialog'`.

Q19. How do you make axe failures readable?

One line per element: rule, impact, selector, help text. Attach the full JSON to the report. Compare the list with an empty array, so the failure message shows every finding at once instead of "expected 0, got 3".

In the project: `expect(result.blocking.map((b) => b.message)).toEqual([])`, with the JSON attached per scan.

Q20. A legacy app has 300 findings. What do you do?

Do not switch rules off. Record the current findings as known, fail only on new ones, and burn the list down by impact (critical and serious first, main journeys first). Each known item needs an owner and a date, otherwise the list only grows.

In the project: the exception file is the small-scale version: ticket, reason, expiry date.

Q21. How do your exceptions work?

`a11y-exceptions.json` lists rule, CSS selector, ticket, reason and expiry date. A finding is excused only if the rule matches and the flagged element matches the selector. After the expiry date it blocks again, with a message naming the expired ticket. An exception that excuses nothing fails a test, so it is removed when the issue is fixed.

In the project: exceptions.ts and its unit tests.

Q22. Why match exceptions by element and not by axe's selector text?

axe reports the shortest selector that is unique on that page: `button` on one page, `button[type="button"]` on another. A text comparison would excuse the issue on some pages and not on others. The helper asks the browser whether the flagged element matches the exception's selector.

In the project: found while building it; the first version compared selector text and failed on every page.

Q23. Why do exceptions expire?

Without a date, a known issue becomes permanent. Expiry forces a decision: fixed, or renewed with a reason someone signs off. It also keeps the accessibility statement honest.

In the project: A11Y-12 expires on 31 March 2027; a unit test checks the day after.

Q24. axe reported a contrast problem that you think is wrong. What do you do?

Check the colours myself (DevTools shows the ratio), and look at the element's real background: transparency, images or a running animation can make axe compute something different from what users see. If axe is wrong, an exception with a reason; if the timing is wrong, fix the test.

In the project: contrast was undecided while the fade-in ran; scans now wait for running animations.

Q25. How do you keep accessibility scans fast?

Scan pages and states, not every step of every test; run them in their own project; reuse the page state the test already reached. A full axe scan takes a fraction of a second to a few seconds per page.

In the project: 15 scans per browser (every page in two colour schemes, three states, the exception check); the whole Chromium accessibility project, keyboard tests included, takes about 15 seconds.

Q26. How do you report accessibility results to a team?

A summary per run: scans, blocking findings, excused findings with their tickets, and what needs a person. Detail in the HTML report. Trends over time if possible.

In the project: a11y-summary.ts writes it into the GitHub job summary.

3. Keyboard and focus

Q27. What can axe not check about keyboard use?

Anything that needs interaction: whether Tab reaches every control in a sensible order, whether focus is visible, whether a dialog traps focus and returns it, whether focus moves to errors. axe sees one moment of the DOM.

In the project: keyboard.ts presses keys and records where focus goes.

Q28. How do you test the tab order automatically?

Open the page, press Tab repeatedly and record the accessible name of each focused element until focus leaves the page or comes round again. Compare with the expected order written in the test. The list doubles as documentation.

In the project: `tabThrough()` and `tabOrderFindings()`; the home page order is written out in the test.

Q29. How do you know when you have reached the end of the page?

Browsers differ. In headless Chromium, focus moves to the browser's own controls (`document.activeElement` becomes the body). Firefox keeps focus on the last element. The helper stops on either, or when the first element comes round again, and identifies elements with a `WeakMap` so two links with the same name are not confused.

In the project: the Firefox behaviour was found when the test first ran there.

Q30. How do you check that focus is visible?

For each tab stop, read the computed style while focused: an outline with a width, or a box shadow. Screenshot comparisons of focus states complement it, because a 1 px outline in the background colour passes a style check.

In the project: `focusIndicatorFindings()` on every page, and the `button-focus.png` baseline.

Q31. What did the focus check find in your own app?

In Chromium, Tab reaches the calendar button inside a date field. That button showed a nearly invisible focus ring, and the field itself no longer matched `:focus`. Styling `input:focus-within` fixed it.

In the project: see "What it found" in the README.

Q32. What makes a dialog accessible?

Focus moves into it when it opens, Tab stays inside, Escape closes it, focus returns to the control that opened it, it has a name (its heading) and the page behind is inert. A native `<dialog>` opened with `showModal()` does most of that; `show()` does not.

In the project: `dialogFindings()`; the `dialog-not-modal` defect swaps `showModal()` for `show()`.

Q33. How do you test a focus trap when focus can leave the page?

With a modal dialog, Tab after the last button may go to the browser's own controls and then back into the dialog. That is fine. What is not fine is focus landing on an element of the page behind. The check allows "no element" but fails on any element outside the dialog.

In the project: two full rounds of Tab inside the dialog, then Escape.

Q34. What is a skip link and how do you test it?

A link at the very top, "Skip to main content", visible when focused, that moves focus to the main content so keyboard users do not tab through the navigation on every page. Test: first Tab focuses it, it is in the viewport, Enter moves focus to `main`, and the next Tab continues inside the main content.

In the project: the skip link test in keyboard.spec.ts.

Q35. Why is a positive tabindex a problem?

`tabindex="1"` puts the element before everything else in the tab order, so the order no longer follows the page. Use `0` to make something focusable in document order and `-1` to focus it only from script.

In the project: the `tab-order-changed` defect; caught by the tab order check and by axe.

Q36. Where should focus go after an action removes the focused element?

Somewhere meaningful and predictable: the page heading, the next item or a status message. If the focused element is simply removed, focus falls back to the body and a screen reader user is lost.

In the project: after cancelling an appointment, focus moves to the `h1` and a status message announces it.

Q37. Can you complete a whole form with the keyboard only in a test?

Yes: focus a field, type, Tab to the next, Space for radios and checkboxes, Enter to submit. Native selects are the exception (keyboard typing varies by browser and OS), so I set them with `selectOption` and say so in a comment.

In the project: "booking form can be completed with the keyboard alone".

4. Forms, errors and dynamic content

Q38. What makes form errors accessible?

The error is in text (not colour only), next to the field, linked with `aria-describedby`, the field has `aria-invalid="true"`, a summary at the top lists all errors as links to the fields, and focus moves to the summary. The page title gets an "Error:" prefix so it is announced when the page changes.

In the project: the GOV.UK pattern in app.js; `errorSummaryFindings()` checks it.

Q39. Why move focus to the error summary?

Without it, a screen reader user presses Submit and hears nothing. The page did change, but nothing tells them. Focus on the summary makes the screen reader read "There is a problem" and the list.

In the project: the `error-summary-not-focused` defect removes one line, and the check catches it.

Q40. When do you use a live region instead of moving focus?

For messages that should be announced without interrupting: "Saved", "Appointment cancelled", search result counts. `role="status"` is polite; `role="alert"` interrupts and is for urgent messages only.

In the project: the cancellation message uses `role="status"`; the test checks its text.

Q41. How do you test that a live region works?

Automated: the region exists before the message is inserted (regions added together with their text are often not announced), it has the right role, and it receives the right text. Whether it is actually announced needs a screen reader.

In the project: the status paragraph is in the page from the start; the test checks role and text.

Q42. Why test with the form in an error state in axe as well?

Error messages add text with colours (red on white must still meet 4.5:1), change `aria-describedby` and can break labels. Those states are never present on page load.

In the project: "booking form with errors shown has no WCAG violations".

Q43. How do you test required fields accessibly?

Required must be visible in text (here: "All fields are required unless marked optional") and programmatic if used (`required` or `aria-required`), and the error must say what to do ("Enter your email address", not "Invalid").

In the project: the error texts are checked word for word in the keyboard test.

Q44. How do you handle a date field?

Native `type="date"` gives an accessible picker in modern browsers, but its parts differ per browser (segments, a calendar button). Give it a hint with the expected format and test its focus states in each browser.

In the project: the date field's calendar button is where the focus problem was found.

5. Contrast, zoom and motion

Q45. What contrast does WCAG AA require?

4.5:1 for normal text, 3:1 for large text (about 24 px, or 18.66 px bold), and 3:1 for user interface components and graphics that carry meaning (1.4.11), including focus indicators.

In the project: the planted `button-contrast` defect is 2.5:1; the vendor badge is 3:1.

Q46. How do you test dark mode?

`colorScheme: 'dark'` in Playwright emulates `prefers-color-scheme: dark`. Scan every page again (all the colours change) and take separate screenshots.

In the project: axe scans and baselines for every page in both schemes.

Q47. How do you test reflow at 400% zoom?

A 1280 px window at 400% zoom is 320 CSS px wide, so test with a 320 px viewport: the page must not scroll horizontally (WCAG 1.4.10). Data tables, maps and diagrams may scroll in their own region.

In the project: `scrollWidth - clientWidth` must be 0 on every page (layout.spec.ts).

Q48. What did the reflow test find?

The appointments page scrolled sideways by 146 px at 320 px. The table scrolled correctly in its region, but the visually hidden button text is absolutely positioned and escaped the region. `position: relative` on the region fixed it.

In the project: a classic bug that is invisible on a normal screen.

Q49. What is the text spacing criterion?

WCAG 1.4.12: no content may be lost when a user sets line height 1.5, paragraph spacing 2 times the font size, letter spacing 0.12 and word spacing 0.16 times the font size. Typical failures are fixed-height boxes with hidden overflow.

In the project: the test applies those values and checks that no element hiding overflow cuts text.

Q50. How do you test reduced motion?

Emulate `prefers-reduced-motion: reduce` and check that animations are off (computed `animation-name: none`, or no running animations). WCAG 2.3.3 is a AAA criterion, but vestibular disorders are common and the fix is one media query.

In the project: the hero fade-in is checked with and without the setting.

6. Visual regression testing

Q51. What is visual regression testing?

Comparing screenshots of the UI with approved baselines and failing on differences. It catches what functional tests miss: spacing, colours, overlapping elements, broken layouts, a missing icon.

In the project: 21 baselines in tests/visual/**screenshots**.

Q52. Why are screenshot tests often flaky, and how do you fix that?

Different rendering between machines (fonts, anti-aliasing, GPU), dynamic content (dates, random ids, ads), animations and carets, and timing (an image not loaded yet). Fixes: render on one platform, fix the clock, mask dynamic parts, disable animations, hide the caret, wait for the page to be ready.

In the project: all of them; see the README section "Why the screenshots are taken in Docker".

Q53. Why render baselines in Docker?

Windows, macOS and Linux render text differently, so a baseline from one never matches a screenshot from another. Rendering in the same image locally and in CI gives identical pixels.

In the project: `npm run visual:docker` uses the Playwright image pinned to the Playwright version; the CI job runs inside the same image.

Q54. What tolerance do you use?

It depends on how deterministic the rendering is. With the same image, browser build and fonts, zero: any difference is real. When baselines must survive small rendering differences, a small `maxDiffPixelRatio` or the per-pixel `threshold`, set as low as works. A high tolerance hides real changes.

In the project: `maxDiffPixels: 0`, after two consecutive identical runs.

Q55. What is the difference between `threshold` and `maxDiffPixels`?

`threshold` (0 to 1) is how different one pixel's colour may be before it counts as different. `maxDiffPixels` and `maxDiffPixelRatio` are how many different pixels the whole image may have.

In the project: default threshold, zero different pixels.

Q56. How do you handle dynamic content?

Make it fixed where you control it (clock, test data) and mask it where you do not (`mask` draws a box over the locator). Masking too much makes the test blind, so prefer fixing.

In the project: the clock is fixed, the random booking reference is masked.

Q57. How do you handle animations?

`animations: 'disabled'` finishes CSS animations and transitions before the screenshot. For JavaScript animations, wait for a stable state or turn them off in the test.

In the project: set in the config for every screenshot.

Q58. Full page or element screenshots?

Full pages catch layout problems anywhere; element screenshots are smaller, more focused and less often affected by unrelated changes. Use full pages for the main pages and states, elements for components and states like hover and focus.

In the project: full pages for pages and states, an element screenshot for hover, a clip for the focus ring.

Q59. How many screenshots are enough?

Each one is a review cost when the design changes. Cover the main pages at the supported breakpoints and themes, and the states that functional tests do not see. Do not screenshot every step of every test.

In the project: 21: 4 pages × 2 sizes × 2 schemes, plus 5 states.

Q60. How are baselines updated?

Only on purpose: `--update-snapshots` for the visual project, in the same Docker image, then the PNG changes are reviewed in the pull request like code. The config sets `updateSnapshots: 'none'` so a missing baseline fails instead of being written silently.

In the project: `npm run visual:update`.

Q61. What does a reviewer look at in a visual diff?

The diff image (changed pixels in red), expected and actual side by side or with a slider. The question is "was this change intended?" A design change: update the baseline. Anything else: a bug.

In the project: the HTML report shows all three images; the README has a screenshot of one.

Q62. Why are your baselines named without browser and platform?

Playwright adds both by default (`home-chromium-linux.png`). There is only one rendering platform here, and the defects project must compare against the same files as the visual project, so the template is `{testDir}/visual/__screenshots__/{arg}{ext}`.

In the project: `snapshotPathTemplate` in playwright.config.ts.

Q63. When would you use a service like Percy, Applitools or Chromatic instead?

When many browsers and devices must be compared, when designers approve changes in a web UI, or when there are hundreds of screenshots. The services render in the cloud (no Docker setup), and some use visual AI to ignore rendering noise. The cost is money and sending screenshots to a third party.

In the project: Playwright's built-in comparison is enough for 21 baselines.

Q64. Do screenshot tests replace functional assertions?

No. A screenshot says "looks the same", not "works". The form test still checks that the confirmation appears; the screenshot checks how it looks.

In the project: the state screenshots first assert the state (focus on the summary, dialog visible).

Q65. How do you screenshot a hover or focus state?

Hover the element (`locator.hover()`), or focus it, then take the screenshot. For a focus ring, clip a little around the element so the ring is inside the picture.

In the project: `button-secondary-hover.png` and `button-focus.png` in states.spec.ts.

Q66. Why skip visual tests on Windows instead of letting them fail?

A failure there would be about the platform, not the app. The skip message says how to run them (`npm run visual:docker`), so nobody wonders why they did not run.

In the project: `onlyWhereBaselinesAreRendered()` in visual.ts.

7. Testing the tests, process and scenarios

Q67. How do you know your accessibility tests work?

Run them against something broken. The project has a regressed build with eight planted defects, and a `defects` project runs the same checks against it. Each defect must produce a specific finding.

In the project: defects.spec.ts; all eight are caught.

Q68. Why a specific finding and not just "the test failed"?

A test can fail for an unrelated reason (a timeout, a changed heading). That would count as "caught" without proving anything. Matching the finding ("focus is not on the error summary") proves the right check caught the right thing.

In the project: each detection has a regular expression the finding must match.

Q69. Why does each check return findings instead of asserting?

So the same implementation can be used both ways: the good build expects an empty list, the regressed build expects a specific item. Two implementations could drift apart.

In the project: every function in keyboard.ts returns `string[]`.

Q70. How do you stop the planted defects from silently disappearing?

Each defect is a text replacement that must match exactly once. If a refactor changes that text, the regressed server throws and a unit test fails, instead of the regressed build quietly becoming the good one.

In the project: server.test.ts checks every defect.

Q71. Which defects did you choose and why?

The ones teams really ship: a missing alt text, a label replaced by a styled paragraph, a lighter brand colour, a removed focus ring, a positive tabindex, a lost `focus()` call, `show()` instead of `showModal()`, and a spacing change. Together they need every kind of check: axe, keyboard, focus management and screenshots.

In the project: defects.ts.

Q72. Why are there no retries?

A check that passes on the second try is a flaky check. Retries would hide it. The visual setup is deterministic and the keyboard checks wait for state, so the suite runs without them.

In the project: `retries: 0` with a comment in the config.

Q73. A product owner asks "is the site accessible?" What do you answer?

What was tested and how: WCAG 2.2 AA rules on these pages and states, keyboard paths, zoom and contrast, with this result, plus the known issues with dates. And what was not: screen reader passes, content quality, user testing. "No automated findings" is not "accessible".

In the project: the accessibility statement and the README's last section.

Q74. A designer wants a light grey placeholder-style text. What do you do?

Show the ratio and the requirement (4.5:1 for text), offer the closest grey that passes, and explain that the test would fail. Fixing it in design is cheaper than an exception later.

In the project: the `button-contrast` defect is that conversation in code.

Q75. A third-party widget fails contrast and the vendor will not fix it this month. What do you do?

Report it to the vendor, add an exception for exactly that rule and element with a ticket and expiry date, name the issue and a workaround in the accessibility statement, and look for a configurable or alternative widget.

In the project: A11Y-12, the chat badge.

Q76. A visual test fails after a dependency update with tiny differences everywhere. What do you do?

Check whether the browser or font rendering changed (a Playwright or image update usually changes anti-aliasing). If the app did not change, re-render the baselines in the new image in their own pull request, reviewed as such, separate from feature changes.

In the project: the image version is pinned to the Playwright version for exactly this reason.

Q77. How would you add accessibility testing to an existing project?

Start with axe on the main pages in CI with known findings recorded, then keyboard tests for the main journeys, then fix by impact. Add `getByRole` locators as tests are touched. Add a manual screen reader pass to the release checklist.

In the project: the pieces are independent and can be copied: `axe.ts`, `exceptions.ts`, `keyboard.ts`.

Q78. What was the most useful thing you learned building this?

That my own "accessible" app had four real problems that only the checks found: a focus ring Chromium hides, hidden text that widened the page at 320 px, contrast measured during an animation, and a selector-based exception that did not hold across pages. Writing the checks found more than reading the code did.

In the project: the README section "What it found in the good app while I built it".